

Schedules in Transactions

Execution Sequences, Serial vs. Concurrent Schedules, Throughput & Consistency

1. What is a Schedule?

A **schedule** is the chronological execution sequence of multiple transactions. It defines the exact order in which operations (reads, writes, commits, or aborts) from various transactions are executed by the DBMS.

Example Sequence: T1 → T2 → T3

2. Classification of Schedules

Schedules are broadly categorized based on how transaction operations are ordered relative to one another:

Serial Schedule

Transactions execute strictly one after another. No interleaving of operations allowed.

Parallel / Concurrent Schedule

Operations of multiple transactions are interleaved, allowing concurrent execution via CPU context switching.

3. Deep Dive: Serial Schedule

In a serial schedule, one transaction must completely finish its execution before the next transaction starts.

★ Golden Rule of Serial Schedules

One transaction at a time: Complete fully first, then start the next. While T1 executes, T2 and T3 remain in a waiting state.

✓ Major Advantage: Consistency

Since transactions execute in isolation without interleaving, interference is impossible. Reasoning about correctness is straightforward.

✗ Major Disadvantage: High Waiting Time

If T1, T2, and T3 arrive together, T2 and T3 experience severe delays waiting for T1 to finish, leading to poor CPU utilization and low throughput.

4. Real-Life Analogy: Single ATM Machine

Imagine a single ATM machine with three people waiting in line (Person A, Person B, Person C):

- **A starts:** Person A uses the ATM while B and C wait in line.
- **A completes:** Person A leaves, then Person B starts using the ATM while C continues waiting.
- **B completes:** Person B leaves, and Person C finally executes their transaction.

*This direct queue mechanism exemplifies a **Serial Schedule**.*

5. Parallel / Concurrent Schedule

In a parallel schedule, operations from multiple transactions are interleaved. Rather than waiting for a transaction to complete, the CPU rapidly switches context between transactions.

Interleaved Order: T1 (Op 1) → T2 (Op 1) → T3 (Op 1) → T1 (Op 2)

 **Important Terminology Note**
"Parallel schedule" in standard DBMS terminology refers to **interleaved CPU execution** across transactions, not necessarily multi-core simultaneous hardware execution.

Real-Life Analogy: Online Banking System

In web banking, thousands of users initiate actions simultaneously (User 1 transfers money, User 2 checks balance, User 3 pays a bill). The system interleaves these operations to keep system resources active rather than freezing all users in a queue.

6. Performance & Throughput

Throughput measures the efficiency of transaction processing in a database engine.

Throughput = Total Transactions Executed / Unit Time
Example: 100 transactions completed in 1 sec = 100 txn/sec

Interleaving ↑ → Waiting Time ↓ → Throughput ↑ → Overall Performance ↑

7. Serial vs. Parallel Schedule (Comparison)

Feature	Serial Schedule	Parallel / Interleaved Schedule
Execution	One transaction at a time	Multiple transactions progress concurrently
Interleaving	 None	 Yes (CPU switching)
Waiting Time	High	Lower
Throughput	Lower	Higher
Consistency	Guaranteed easily	Requires Concurrency Control (Locks, etc.)

Memory Tricks & Exam Essentials

 **Super-Easy Memory Trick**

- Serial = Single Queue** (One completes → next starts)
- Parallel = Context Switch** (CPU switches among tasks)

 **Golden Summary Line**
"Serial ensures easy consistency at the cost of waiting time; Parallel maximizes throughput by interleaving, but requires concurrency control."